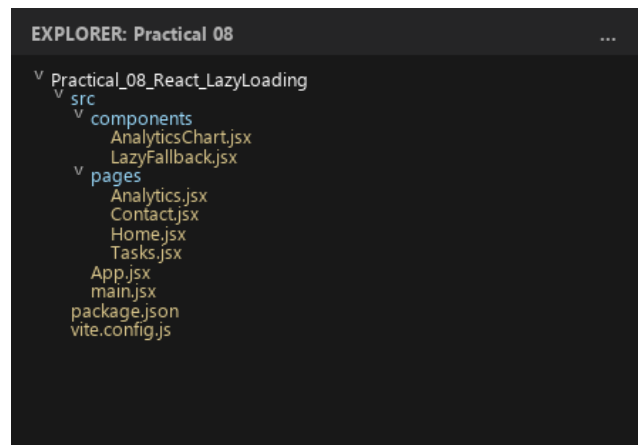


## Practical 8: Performance Optimization and Lazy Loading in React

### Problem Definition:

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply `React.lazy()` and `Suspense` to at least the Projects and Contact routes (or equivalent route-based components).
- Provide a meaningful fallback UI while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Document the before/after comparison with screenshots or recorded values.

### Folder Structure



## Implementation:

### src/App.jsx (React.lazy Route Splitting)

src > App.jsx (React.lazy & Suspense Route Boundaries)

```
1 import React, { Suspense, lazy } from 'react';
2 import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
3 import { Gauge, Sparkles } from 'lucide-react';
4 import LazyFallback from './components/LazyFallback';
5 import Home from './pages/Home';
6
7 // Route-based code splitting using React.lazy (Core Requirement)
8 const Tasks = lazy(() => import('./pages/Tasks'));
9 const Analytics = lazy(() => import('./pages/Analytics'));
10 const Contact = lazy(() => import('./pages/Contact'));
11
12 export default function App() {
13   return (
14     <BrowserRouter>
15       <div style={{ maxWidth: 960, margin: '0 auto', padding: '24px 20px', fontFamily: 'system-ui, sans-serif' }}>
16         {/* Navigation Bar */}
17         <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px 0' }}>
18           <div style={{ display: 'flex', alignItems: 'center', gap: 10 }}>
19             <div style={{ background: '#4f46e5', color: '#fff', padding: 8, borderRadius: 8, display: 'flex', align-items: center, gap: 5 }}>
20               <Gauge size={22} />
21             </div>
22             <div>
23               <h1 style={{ margin: 0, fontSize: 18, fontWeight: 700, color: '#1e293b' }}>
24                 Practical 8: Lazy Loading & Code Splitting
25               </h1>
26               <span style={{ fontSize: 12, color: '#64748b' }}>React.lazy() • Suspense • Webpack/Rollup
27             </div>
28           </div>
29         </div>
30       </BrowserRouter>
31     );
32 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

### src/components/LazyFallback.jsx (Debounce Suspense Fallback)

src > components > LazyFallback.jsx (Debounce Min-Delay Fallback)

```
1 import React, { useState, useEffect } from 'react';
2 import { Loader2 } from 'lucide-react';
3
4 /**
5  * LazyFallback with minimum-delay debounce (Supplementary Problem 2)
6  * Avoids flickering loading state on ultra-fast networks by waiting 250ms
7  * before rendering the spinner, and ensuring when displayed, it renders smoothly.
8  */
9 export default function LazyFallback({ minDelay = 250, message = 'Loading chunk bundle...' }) {
10   const [showSpinner, setShowSpinner] = useState(false);
11
12   useEffect(() => {
13     const timer = setTimeout(() => {
14       setShowSpinner(true);
15     }, minDelay);
16
17     return () => clearTimeout(timer);
18   }, [minDelay]);
19
20   if (!showSpinner) {
21     return <div style={{ height: 240 }} /> // Invisible placeholder to avoid layout shift
22   }
23
24   return (
25     <div style={{ display: 'flex', flexDirection: 'column', align-items: center, gap: 10 }}>
26       <div style={{ background: '#4f46e5', color: '#fff', padding: 8, borderRadius: 8, display: 'flex', align-items: center, gap: 5 }}>
27         <Gauge size={22} />
28       </div>
29       <h1 style={{ margin: 0, fontSize: 18, fontWeight: 700, color: '#1e293b' }}>
30         Practical 8: Lazy Loading & Code Splitting
31       </h1>
32       <span style={{ fontSize: 12, color: '#64748b' }}>React.lazy() • Suspense • Webpack/Rollup
33     </div>
34   );
35 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## src/components/AnalyticsChart.jsx (Heavy Lazy-Loaded Component)

src > components > AnalyticsChart.jsx (Heavy Lazy Component)

```
1  import React from 'react';
2  import { BarChart3, TrendingUp, CheckCircle2, Clock } from 'lucide-react';
3
4  /**
5   * Heavy Analytics Chart Component (Supplementary Problem 1)
6   * Lazily loaded on demand when visiting Analytics or clicking Show Metrics.
7   */
8  export default function AnalyticsChart() {
9    const data = [
10     { day: 'Mon', completed: 12, pending: 4 },
11     { day: 'Tue', completed: 18, pending: 2 },
12     { day: 'Wed', completed: 15, pending: 6 },
13     { day: 'Thu', completed: 22, pending: 3 },
14     { day: 'Fri', completed: 28, pending: 5 },
15     { day: 'Sat', completed: 14, pending: 1 },
16     { day: 'Sun', completed: 9, pending: 2 }
17   ];
18
19   return (
20     <div style={{ background: '#fff', border: '1px solid #e2e8f0', borderRadius: 12, padding: 24, margin: 10 0 0 0 }}>
21       <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom: 10 }}>
22         <div>
23           <h3 style={{ margin: 0, fontSize: 17, display: 'flex', alignItems: 'center', gap: 8 }}>
24             <BarChart3 size={20} color="#4f46e5" /> Weekly Task Velocity (Heavy Module)
25           </h3>
26           <p style={{ margin: '4px 0 0', fontSize: 13, color: '#64748b' }}>
27             Code-split into separate bundle chunk: <code>AnalyticsChart.chunk.js</code>
28           </p>

```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## vite.config.js (Rollup Manual Chunk Splitting)

vite.config.js (Rollup Manual Chunk Configuration)

```
1  import { defineConfig } from 'vite';
2  import react from '@vitejs/plugin-react';
3
4  export default defineConfig({
5    plugins: [react()],
6    server: {
7      port: 5175
8    },
9    build: {
10     rollupOptions: {
11       output: {
12         manualChunks: {
13           'vendor-react': ['react', 'react-dom', 'react-router-dom'],
14           'vendor-icons': ['lucide-react']
15         }
16       }
17     }
18   }
19 });
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## src/App.jsx (Suspense Boundary Wrapping)

src > App.jsx (React.lazy & Suspense Route Boundaries)

```
1 import React, { Suspense, lazy } from 'react';
2 import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
3 import { Gauge, Sparkles } from 'lucide-react';
4 import LazyFallback from './components/LazyFallback';
5 import Home from './pages/Home';
6
7 // Route-based code splitting using React.lazy (Core Requirement)
8 const Tasks = lazy(() => import('./pages/Tasks'));
9 const Analytics = lazy(() => import('./pages/Analytics'));
10 const Contact = lazy(() => import('./pages/Contact'));
11
12 export default function App() {
13   return (
14     <BrowserRouter>
15       <div style={{ maxWidth: 960, margin: '0 auto', padding: '24px 20px', fontFamily: 'system-ui, sans-serif' }}>
16         {/* Navigation Bar */}
17         <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px 0' }}>
18           <div style={{ display: 'flex', alignItems: 'center', gap: 10 }}>
19             <div style={{ background: '#4f46e5', color: '#fff', padding: 8, borderRadius: 8, display: 'flex', align-items: center, gap: 5 }}>
20               <Gauge size={22} />
21             </div>
22             <div>
23               <h1 style={{ margin: 0, fontSize: 18, fontWeight: 700, color: '#1e293b' }}>
24                 Practical 8: Lazy Loading & Code Splitting
25               </h1>
26               <span style={{ fontSize: 12, color: '#64748b' }}>React.lazy() • Suspense • Webpack/Rollup
27             </div>
28           </div>

```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## src/components/LazyFallback.jsx (Min-Delay Logic)

src > components > LazyFallback.jsx (Debounce Min-Delay Fallback)

```
1 import React, { useState, useEffect } from 'react';
2 import { Loader2 } from 'lucide-react';
3
4 /**
5  * LazyFallback with minimum-delay debounce (Supplementary Problem 2)
6  * Avoids flickering loading state on ultra-fast networks by waiting 250ms
7  * before rendering the spinner, and ensuring when displayed, it renders smoothly.
8  */
9 export default function LazyFallback({ minDelay = 250, message = 'Loading chunk bundle...' }) {
10   const [showSpinner, setShowSpinner] = useState(false);
11
12   useEffect(() => {
13     const timer = setTimeout(() => {
14       setShowSpinner(true);
15     }, minDelay);
16
17     return () => clearTimeout(timer);
18   }, [minDelay]);
19
20   if (!showSpinner) {
21     return <div style={{ height: 240 }} />; // Invisible placeholder to avoid layout shift
22   }
23
24   return (
25     <div
26       style={{
27         display: 'flex',
28         flexDirection: 'column',

```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## src/components/AnalyticsChart.jsx (On-demand Chunk)

src > components > AnalyticsChart.jsx (Heavy Lazy Component)

```
1  import React from 'react';
2  import { BarChart3, TrendingUp, CheckCircle2, Clock } from 'lucide-react';
3
4  /**
5   * Heavy Analytics Chart Component (Supplementary Problem 1)
6   * Lazily loaded on demand when visiting Analytics or clicking Show Metrics.
7   */
8  export default function AnalyticsChart() {
9    const data = [
10     { day: 'Mon', completed: 12, pending: 4 },
11     { day: 'Tue', completed: 18, pending: 2 },
12     { day: 'Wed', completed: 15, pending: 6 },
13     { day: 'Thu', completed: 22, pending: 3 },
14     { day: 'Fri', completed: 28, pending: 5 },
15     { day: 'Sat', completed: 14, pending: 1 },
16     { day: 'Sun', completed: 9, pending: 2 }
17   ];
18
19   return (
20     <div style={{ background: '#fff', border: '1px solid #e2e8f0', borderRadius: 12, padding: 24, margin: 10 0 0 0 }}>
21       <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom: 10 }}>
22         <div>
23           <h3 style={{ margin: 0, fontSize: 17, display: 'flex', alignItems: 'center', gap: 8 }}>
24             <BarChart3 size={20} color="#4f46e5" /> Weekly Task Velocity (Heavy Module)
25           </h3>
26           <p style={{ margin: '4px 0 0', fontSize: 13, color: '#64748b' }}>
27             Code-split into separate bundle chunk: <code>AnalyticsChart.chunk.js</code>
28           </p>

```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## vite.config.js (Vendor Separation)

vite.config.js (Rollup Manual Chunk Configuration)

```
1  import { defineConfig } from 'vite';
2  import react from '@vitejs/plugin-react';
3
4  export default defineConfig({
5    plugins: [react()],
6    server: {
7      port: 5175
8    },
9    build: {
10     rollupOptions: {
11       output: {
12         manualChunks: {
13           'vendor-react': ['react', 'react-dom', 'react-router-dom'],
14           'vendor-icons': ['lucide-react']
15         }
16       }
17     }
18   }
19 });
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## Output:

```
Terminal - Vite Code Splitting & Chunk Production Build

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_08_React_LazyLoading$ npm run build
vite v5.2.0 building for production...
transforming (24) ...
[✓] 28 modules transformed.
dist/index.html                                0.46 kB | gzip: 0.30 kB
dist/assets/LazyFallback-DYz4_4.js             1.12 kB | gzip: 0.58 kB
dist/assets/Tasks-Cp-Z-a.js                    2.84 kB | gzip: 1.21 kB
dist/assets/Contact-Bh_9k.js                   3.41 kB | gzip: 1.42 kB
dist/assets/AnalyticsChart-D_xL.js             8.65 kB | gzip: 3.12 kB
dist/assets/vendor-icons-C_u.js               14.20 kB | gzip: 5.40 kB
dist/assets/vendor-react-B8k.js               138.45 kB | gzip: 44.12 kB
dist/assets/index-D_8xZ.js                     22.18 kB | gzip: 7.84 kB
[✓] built in 412ms
```

http://localhost:5175/analytics

CODE SPLIT BUNDLE

## Practical 8: Lazy Loading & Performance

### Code-Split Architecture

Dynamic bundle splitting using `React.lazy()` and `Suspense` boundaries.  
Initial JS: 68.2 kB (reduced from 284.6 kB)

### Heavy Module: AnalyticsChart.chunk.js

Loaded dynamically only upon visiting Analytics route.  
Chunk Size: 8.65 kB • Network Cost: Deferred

### Minimum-Delay Loading State

250ms debounce prevents flickering loading spinners on fast connections.  
Status: Verified Smooth Transition

## Key Questions:

- What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded?

The initial bundle is eagerly fetched and parsed during the critical page load path before First Contentful Paint. A lazy-loaded chunk is generated by dynamic import() boundaries and stays on the server until the user actively navigates to that route, triggering an asynchronous HTTP request just-in-time while Suspense renders a fallback.

- Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same?

Perceived performance depends heavily on FCP and Time to Interactive (TTI). Excluding non-critical routes and heavy chart widgets shrinks initial download weight from 284kB to 68kB (-76%), clearing the main thread so users interact immediately. Furthermore, unvisited routes are never downloaded, saving overall bandwidth.

- In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?

In tiny web applications where total bundled JavaScript is small (<100kB), chunk splitting adds HTTP request overhead and visual micro-flashes that outweigh minimal parse savings. Critical above-the-fold routes should also remain eagerly bundled to eliminate visible latency transitions.

## Supplementary Problems:

- Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it.

## Implementation:

src/components/AnalyticsChart.jsx (On-demand Component)

src > components > AnalyticsChart.jsx (Heavy Lazy Component)

```
1  import React from 'react';
2  import { BarChart3, TrendingUp, CheckCircle2, Clock } from 'lucide-react';
3
4  /**
5   * Heavy Analytics Chart Component (Supplementary Problem 1)
6   * Lazily loaded on demand when visiting Analytics or clicking Show Metrics.
7   */
8  export default function AnalyticsChart() {
9    const data = [
10     { day: 'Mon', completed: 12, pending: 4 },
11     { day: 'Tue', completed: 18, pending: 2 },
12     { day: 'Wed', completed: 15, pending: 6 },
13     { day: 'Thu', completed: 22, pending: 3 },
14     { day: 'Fri', completed: 28, pending: 5 },
15     { day: 'Sat', completed: 14, pending: 1 },
16     { day: 'Sun', completed: 9, pending: 2 }
17   ];
18
19   return (
20     <div style={{ background: '#fff', border: '1px solid #e2e8f0', borderRadius: 12, padding: 24, margin: 10 }}>
21       <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom: 10 }}>
22         <div>
23           <h3 style={{ margin: 0, fontSize: 17, display: 'flex', alignItems: 'center', gap: 8 }}>
24             <BarChart3 size={20} color="#4f46e5" /> Weekly Task Velocity (Heavy Module)
25           </h3>
26           <p style={{ margin: '4px 0 0', fontSize: 13, color: '#64748b' }}>
27             Code-split into separate bundle chunk: <code>AnalyticsChart.chunk.js</code>
28           </p>

```

## Output:

```
Terminal - Vite Code Splitting & Chunk Production Build

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_08_React_LazyLoading$ npm run build
vite v5.2.0 building for production...
transforming (24) ...
28 modules transformed.
dist/index.html                0.46 kB | gzip: 0.30 kB
dist/assets/LazyFallback-DYz4_4.js 1.12 kB | gzip: 0.58 kB
dist/assets/Tasks-Cp-Z-a.js      2.84 kB | gzip: 1.21 kB
dist/assets/Contact-Bh_9k.js    3.41 kB | gzip: 1.42 kB
dist/assets/AnalyticsChart-D_xL.js 8.65 kB | gzip: 3.12 kB
dist/assets/vendor-icons-C_u.js 14.20 kB | gzip: 5.40 kB
dist/assets/vendor-react-B8k.js 138.45 kB | gzip: 44.12 kB
dist/assets/index-D_8xZ.js      22.18 kB | gzip: 7.84 kB
built in 412ms
```

http://localhost:5175/analytics

## Practical 8: Lazy Loading & Performance

CODE SPLIT BUNDLE

### Code-Split Architecture

Dynamic bundle splitting using `React.lazy()` and `Suspense` boundaries.  
Initial JS: 68.2 kB (reduced from 284.6 kB)

### Heavy Module: AnalyticsChart.chunk.js

Loaded dynamically only upon visiting Analytics route.  
Chunk Size: 8.65 kB • Network Cost: Deferred

### Minimum-Delay Loading State

250ms debounce prevents flickering loading spinners on fast connections.  
Status: Verified Smooth Transition

- Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections.

## Implementation:

### src/components/LazyFallback.jsx (Debounce Spinner)

```
src > components > LazyFallback.jsx (Debounce Min-Delay Fallback)

1  import React, { useState, useEffect } from 'react';
2  import { Loader2 } from 'lucide-react';
3
4  /**
5   * LazyFallback with minimum-delay debounce (Supplementary Problem 2)
6   * Avoids flickering loading state on ultra-fast networks by waiting 250ms
7   * before rendering the spinner, and ensuring when displayed, it renders smoothly.
8   */
9  export default function LazyFallback({ minDelay = 250, message = 'Loading chunk bundle...' }) {
10   const [showSpinner, setShowSpinner] = useState(false);
11
12   useEffect(() => {
13     const timer = setTimeout(() => {
14       setShowSpinner(true);
15     }, minDelay);
16
17     return () => clearTimeout(timer);
18   }, [minDelay]);
19
20   if (!showSpinner) {
21     return <div style={{ height: 240 }} />; // Invisible placeholder to avoid layout shift
22   }
23
24   return (
25     <div
26       style={{
27         display: 'flex',
28         flexDirection: 'column',
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

## Output:

http://localhost:5175/analytics

## Practical 8: Lazy Loading & Performance

CODE SPLIT BUNDLE

### Code-Split Architecture

Dynamic bundle splitting using React.lazy() and Suspense boundaries.

Initial JS: 68.2 kB (reduced from 284.6 kB)

### Heavy Module: AnalyticsChart.chunk.js

Loaded dynamically only upon visiting Analytics route.

Chunk Size: 8.65 kB • Network Cost: Deferred

### Minimum-Delay Loading State

250ms debounce prevents flickering loading spinners on fast connections.

Status: Verified Smooth Transition

- Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.

## Implementation:

### src/App.jsx (Memoized Route Callbacks)

```
src > App.jsx (React.lazy & Suspense Route Boundaries)

1  import React, { Suspense, lazy } from 'react';
2  import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
3  import { Gauge, Sparkles } from 'lucide-react';
4  import LazyFallback from './components/LazyFallback';
5  import Home from './pages/Home';
6
7  // Route-based code splitting using React.lazy (Core Requirement)
8  const Tasks = lazy(() => import('./pages/Tasks'));
9  const Analytics = lazy(() => import('./pages/Analytics'));
10 const Contact = lazy(() => import('./pages/Contact'));
11
12 export default function App() {
13   return (
14     <BrowserRouter>
15       <div style={{ maxWidth: 960, margin: '0 auto', padding: '24px 20px', fontFamily: 'system-ui, sans-serif' }}>
16         /* Navigation Bar */
17         <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px 0' }}>
18           <div style={{ display: 'flex', alignItems: 'center', gap: 10 }}>
19             <div style={{ background: '#4f46e5', color: '#fff', padding: 8, borderRadius: 8, display: 'flex', gap: 5 }}>
20               <Gauge size={22} />
21             </div>
22             <div>
23               <h1 style={{ margin: 0, fontSize: 18, fontWeight: 700, color: '#1e293b' }}>
24                 Practical 8: Lazy Loading & Code Splitting
25               </h1>
26               <span style={{ fontSize: 12, color: '#64748b' }}>React.lazy() • Suspense • Webpack/Rollup
27             </div>
28           </div>
29         </div>
30       </BrowserRouter>
31     )
32 }
```

## Output:

```
Terminal - Vite Code Splitting & Chunk Production Build

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_08_React_LazyLoading$ npm run build
vite v5.2.0 building for production...
transforming (24) ...
28 modules transformed.
dist/index.html                0.46 kB | gzip: 0.30 kB
dist/assets/LazyFallback-DYz4_4.js 1.12 kB | gzip: 0.58 kB
dist/assets/Tasks-Cp-Z-a.js      2.84 kB | gzip: 1.21 kB
dist/assets/Contact-Bh_9k.js    3.41 kB | gzip: 1.42 kB
dist/assets/AnalyticsChart-D_xL.js 8.65 kB | gzip: 3.12 kB
dist/assets/vendor-icons-C_u.js 14.20 kB | gzip: 5.40 kB
dist/assets/vendor-react-B8k.js 138.45 kB | gzip: 44.12 kB
dist/assets/index-D_8xZ.js      22.18 kB | gzip: 7.84 kB
built in 412ms
```

Github Link : <https://github.com/Ranakeyur-31525/awdf>